BAŞLIK: **Naive toplamanın doğruluğu: hatanın büyümesi, koşul sayısı ve daha iyi yöntemler**
KONU: `Naive-Summation` deposunun 2. sürümündeki analizlerin amacı, yöntemi ve sonuçları
YAZAR: Muhammed İkbal Yılmaz (myucanlar@gmail.com)

1. Amaç ve kapsam

Naive (ardışık) toplama $S = \sum_{i=1}^{n} x_i$ toplamını $s_0 = 0$, $s_k = s_{k-1} + x_k$ biçiminde hesaplar. Kayan noktalı aritmetikte her toplama yuvarlanır: $\mathrm{fl}(a + b) = (a + b)(1 + \delta)$, $|\delta| \le \varepsilon$. Burada $\varepsilon = 2^{-53} \approx 1{,}1 \times 10^{-16}$, çift duyarlıklı sayıların birim yuvarlama hatasıdır. Bu yüzden hesaplanan toplam $\hat{S}$, $E = \hat{S} - S$ kadar hatalıdır. Projenin 1. sürümü rastgele veriler için E'yi ölçmüş ve naive toplamanın "sayısal olarak kararsız ve güvenilmez" olduğu sonucuna varmıştı. Bu not daha dar üç soru soruyor; her birini bir analiz yanıtlıyor:

1. **Hata n ile nasıl büyüyor?** Teorik sınırla karşılaştırıldığında durum nedir ve bu, veriye bağlı mı?
2. **Göreli hatayı $|E|/|S|$ ne büyütüyor?** 1. sürüm bu hatanın "dalgalandığını" bulmuştu.
3. **Daha iyi yöntemler ne kazandırıyor, bedeli ne?** 1. sürümün referans toplamı ne kadar doğruydu?

2. Yöntem

- **Kesin hata.** Her hata kesin olarak hesaplanır: $\hat{S}$ ve $-x_1, \ldots, -x_n$, toplamı ara yuvarlama yapmadan izleyen `math.fsum` ile toplanır ve sonuç yalnızca bir kez yuvarlanır. Yuvarlanmış bir referans toplamla karşılaştırmak her hataya son basamağın yarısı kadar bir hata ekler; bu da en iyi yöntemlerin hatası büyüklüğündedir. Birim testleri bu hesabı kesin rasyonel aritmetikle karşılaştırır.
- **Yöntemler.** Naive toplama (`np.cumsum` ile hesaplanır; testler bunun Python döngüsüyle bit bit aynı sonucu verdiğini ve tüm ara toplamları tek seferde ürettiğini gösterir); komşu terimleri kademe kademe toplayan ikili (pairwise) toplama; ve her toplamada kaybolan düşük basamaklı kısmı bir sonrakine taşıyan Kahan'ın (1965) telafili toplaması.
- **Sınırlar** (Higham, 2002, 4. bölüm), $\gamma_k = k\varepsilon/(1 - k\varepsilon)$ olmak üzere: naive $|E| \le \gamma_{n-1} \sum |x_i|$; ikili $|E| \le \gamma_{\lceil \log_2 n \rceil} \sum |x_i|$; Kahan $|E| \le (2\varepsilon + O(n\varepsilon^2)) \sum |x_i|$. Birim testlerinde ölçülen her hata bu sınırların içinde kalır.
- **Veri.** 1. sürümdeki gibi $\mathrm{U}(-1, 1)$ tekdüze rastgele sayılar;

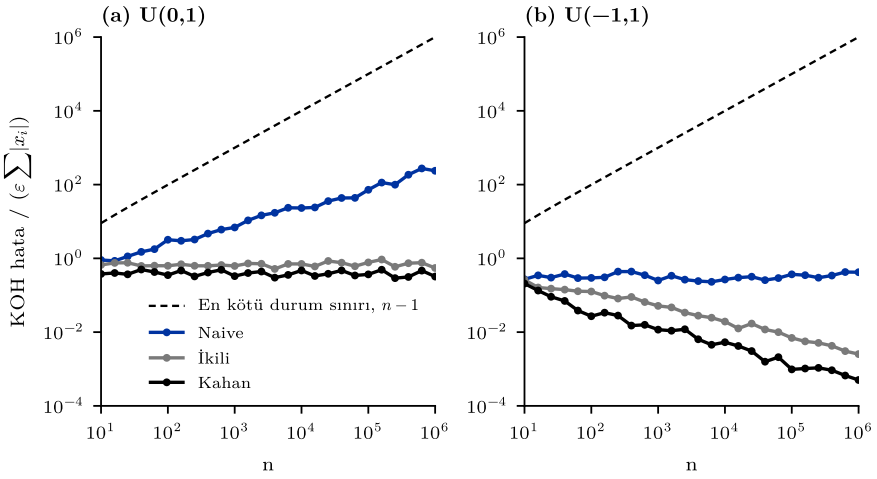
karşılaştırma için tüm terimlerin aynı işaretli olduğu $\mathrm{U}(0,1)$. Tohumlar sabittir.

3. Analiz 1: Hatanın büyümesi (Şekil 1)

Neden. Naive toplamanın sınırı n ile doğrusal büyür; 1. sürüm de hatanın doğrusal büyüdüğünü yazmıştı. Ancak sınır en kötü durumu anlatır: her yuvarlama hatasının en büyük boyda ve aynı işaretle oluştuğunu varsayar. Soru, hatanın pratikte nasıl büyüdüğüdür.

Yöntem. Her dağılım için 10^6 elemanlı 30 dizi alındı; 10 ile 10^6 arasındaki 26 uzunluk n için ilk n elemanın toplam hatası kesin olarak hesaplandı. Şekil, $|E|/(\varepsilon \sum |x_i|)$'nin kök ortalama karesini (KOH) gösterir; bu birimde naive toplamanın sınırı $n-1$ doğrusudur.

Şekil nasıl okunur? İki eksen de logaritmiktir; n^k ile büyüyen bir büyüklük eğimi k olan bir doğru olarak görünür. Bir eğri ile kesikli çizgi arasındaki uzaklık, sınırın ne kadar karamsar olduğunu gösterir.



Şekil 1. Üç yöntemin 30 dizi üzerinden KOH hatası, $\varepsilon \sum |x_i|$ biriminde. Kesikli çizgi: naive toplamanın en kötü durum sınırı.

Bulgu. Pozitif terimlerde, $\mathrm{U}(0,1)$, naive hata bu ölçekte 0,50 eğimle büyür: yuvarlama hataları birbirinden bağımsız rastgele işaretli gibi davrandığında öngörüldüğü gibi $\sqrt{n}$ hızında (Johnson, 2019). $n = 10^6$'da hata 237 birimdir; sınır ise 10^6'dır. İki işaretli terimlerde, $\mathrm{U}(-1,1)$, eğim yalnızca 0,06'dır: ara toplamlar küçük kaldığı için her yuvarlama hatası da küçüktür ve hata bir birimin altında kalır ($n = 10^6$'da 0,42). Her iki durumda da ikili ve Kahan toplaması her n için bir birimin altındadır ($n = 10^6$'da $\mathrm{U}(0,1)$ için 0,55 ve 0,32; $\mathrm{U}(-1,1)$ için 0,0025 ve 0,0005).

Yorum. "Hata doğrusal büyür" ifadesi tipik davranışı değil, sınırı anlatır. Hatanın gerçekte ne hızla büyüdüğü, ara toplamların büyüklüğü

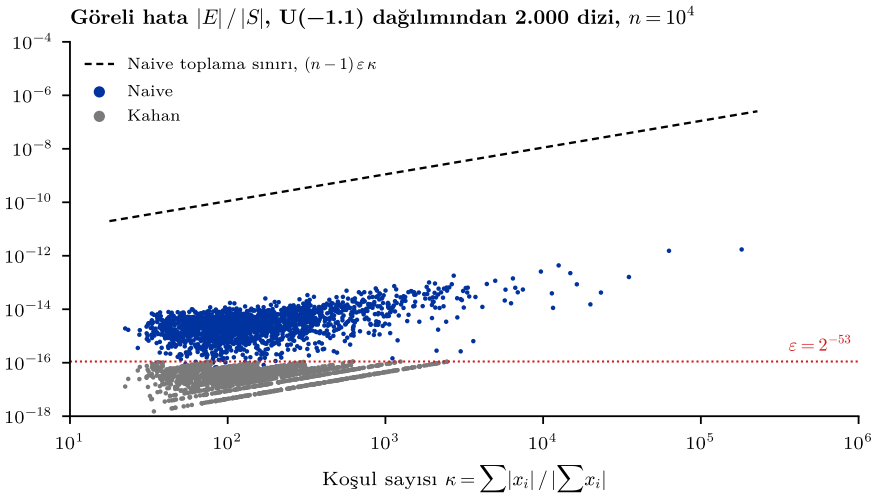
aracılığıyla veriye bağlıdır.

4. Analiz 2: Koşul sayısı (Şekil 2)

Neden. 1. sürümde göreli hata çalıştırmadan çalıştırmaya çok değişiyordu. Naive sınır $|S|$'ye bölününce $|E|/|S| \leq \gamma_{n-1}\,\kappa$ elde edilir; $\kappa = \sum |x_i| / |\sum x_i|$ toplamın koşul sayısıdır. İki işaretli terimler birbirini neredeyse götürdüğünde $|S|$ küçük, κ büyük olur; böylece küçük bir mutlak hata bile büyük bir göreli hataya dönüşür. Bu analiz, değişkenliği κ'nın açıklayıp açıklamadığını sınar.

Yöntem. U$(-1, 1)$'den $n = 10^4$ elemanlı 2.000 dizi; her biri için κ ile naive ve Kahan toplamının kesin göreli hatası.

Şekil nasıl okunur? Her nokta bir dizidir. Noktaların sağa doğru yükselmesi, göreli hatanın koşul sayısını izlediği anlamına gelir.



Şekil 2. U$(-1, 1)$'den 10^4 elemanlı 2.000 dizi için göreli hata ve koşul sayısı. Kesikli çizgi: naive toplama sınırı. Noktalı çizgi: ε.

Bulgu. κ'nın medyanı 125, en büyük değeri 181.407'dir. Naive göreli hata κ ile yükselir (log–log ölçekte eğim 0,64) ve sınırın çok altında kalır (0 ihlal). Ortalaması, $7{,}6 \times 10^{-15}$, medyanının ($2{,}7 \times 10^{-15}$) 2,8 katıdır; çünkü en büyük hatalı %1'lik dilim ortalamanın %37'sini oluşturur: 1. sürümdeki gibi göreli hataların ortalaması, sıfıra yakın birkaç toplamın etkisi altındadır. Kahan'ın göreli hatası κ ne olursa olsun $1{,}00\,\varepsilon$'u aşmaz; noktalarının bantlar oluşturmasının nedeni, sonucun kesin toplamdan en fazla bir yuvarlama uzakta olması ve yuvarlamanın kesikli adımlarla gerçekleşmesidir.

Yorum. "Dalgalanma" yöntemin kararsızlığı değil, problemin koşulluluğudur. Naive toplama geriye doğru kararlıdır: $\hat{S}$, girdilerin en fazla γ_{n-1} oranında bozulmuş hâllerinin kesin toplamıdır (Higham,

2002, 4. bölüm; Johnson, 2019). Kötü koşullu bir toplamda büyük göreli hata, geriye doğru kararlı her yöntemin üretebileceği bir sonuçtur.

5. Analiz 3: Yöntemler ve 1. sürümün referansı (Tablo 1)

Neden. Daha doğru bir yöntem, ancak maliyeti kabul edilebilirse kullanılmaya değer; bu yüzden doğruluk ve süre birlikte ölçüldü. 1. sürüm kesin toplam olarak `np.sum(x, dtype=np.longdouble)` kullanmıştı; `longdouble`'ın duyarlılığı platformdan platforma değişir (NumPy belgeleri). Bu nedenle onun doğruluğu da ölçüldü.

Yöntem. 1. sürümdeki gibi $\mathrm{U}(-1,1)$'den 10^6 elemanlı 10 dizi. Hatalar 1. analizdeki gibi; süre beş çağrının medyanıdır (Darwin arm64, Python 3.13.6, NumPy 2.4.1).

Tablo 1. $n = 10^6$ için $\varepsilon \sum |x_i|$ biriminde KOH hata ve çalışma süresi.

Yöntem	KOH hata	Süre (ms)
Naive, Python döngüsü (sürüm 1)	0,442	17,86
Naive, `np.cumsum`	0,442	2,09
İkili	$1{,}9 \times 10^{-3}$	0,47
`np.sum` (NumPy, kısmi ikili)	$2{,}4 \times 10^{-3}$	0,13
Kahan, Python döngüsü	$4{,}2 \times 10^{-4}$	67,10
`math.fsum` (doğru yuvarlanmış)	$4{,}2 \times 10^{-4}$	15,13
`np.sum`, `longdouble` (sürüm 1 referansı)	$2{,}4 \times 10^{-3}$	0,13

Bulgu. 1. sürümün Python döngüsü hem isabetsiz hem yavaştır: 17,9 ms sürer. Sonucu kesin toplamın doğru yuvarlanmış hâli olan `math.fsum` ise 15,1 ms sürer (1,2 kat hızlı); `np.sum` de 0,13 ms'de, iki basamak daha küçük bir hatayla sonuç verir. Kahan yöntemi `fsum`'un doğruluğuna ulaşır ama Python'da yazıldığında yavaştır (67 ms); avantajı, terim başına yalnızca sabit miktarda ek iş gerektirmesidir ve bu, derlenmiş kodda önemlidir. Bu makinede `longdouble`, `float64` ile aynı duyarlılıktadır; dolayısıyla sürüm 1'in referansı aslında NumPy'ın çift duyarlıklı ikili toplamıdır; hatası naive toplama hatasının %0,5 kadarıdır. Burada yapıldığı gibi hataları kesin bir referansla ölçmek bu platform bağımlılığını ortadan kaldırır.

6. Sonuç ve sınırlılıklar

- Naive toplamanın hatası en kötü durum sınırının çok altındadır. Aynı işaretli terimlerde sınırın ölçeğinde $\sqrt{n}$ hızında büyür, birbirini götüren terimlerde ise neredeyse hiç büyümez. Göreli hatayı koşul sayısı belirler.
- Naive toplama kararsız değil, geriye doğru kararlıdır. Python'da doğru toplam için `math.fsum` hem doğru yuvarlanmış sonuç verir hem de Python döngüsünden hızlıdır; `np.sum` hızlıdır ve naive toplamadan çok daha doğrudur.

- Yalnızca tekdüze veri, çift duyarlılık ve 10^6'ya kadar boyutlar incelendi. Süreler tek bir makinede ölçüldü.

Kaynaklar

Higham, N. J. (1993). The accuracy of floating point summation. *SIAM Journal on Scientific Computing*, 14(4), 783–799.

Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms* (2. baskı). SIAM.

Johnson, S. G. (2019). Notes on the accuracy of naive summation; Backwards stability of recursive summation. MIT 18.335J, *Introduction to Numerical Methods*, 3. ders.

Kahan, W. (1965). Further remarks on reducing truncation errors. *Communications of the ACM*, 8(1), 40.

NumPy belgeleri: `numpy.sum`, `numpy.cumsum`, `numpy.finfo`. Python belgeleri: `math.fsum`.

Tüm sayılar `figures/*.csv` dosyalarından `make_paper.py` ile üretilir.